Focal Tversky Loss and Segmentation Metrics for Wildfire Detection

Introduction

Your GTX 1660 SUPER completed the forward pass allocating only 124.33 MB of VRAM, confirming that your batch size of 8 fits within your 6 GB hardware ceiling with ample headroom for backpropagation and optimizer states.

Hardware Memory Budget Analysis

Understanding how GPU memory is consumed during deep learning prevents out-of-memory (`CUDA: out of memory`) crashes:

- **Static Model Weights:** $31,037,633$ parameters stored as 32-bit single-precision floats (4 bytes per parameter) consume exactly $31,037,633 \times 4 \approx 124.15$ MB.
- **Gradient Tensors:** During backpropagation, PyTorch calculates and stores the partial derivative $\frac{\partial L}{\partial W}$ for every single parameter, allocating an identical 124.15 MB.
- **AdamW Optimizer States:** The AdamW optimizer tracks two running statistics for every parameter—the first moment vector (m_t, exponential moving average of gradients) and the second moment vector (v_t, moving average of squared gradients). This requires 2×124.15 MB ≈ 248.3 MB.
- **Activation Caching:** During the forward pass, intermediate feature maps from all 23 convolutional layers are held in memory so the chain rule can reference them during the backward pass. For a batch of 8 images at 128×256, this consumes approximately 350 to 450 MB.

Your total active GPU footprint during full training will remain under 1.1 GB, leaving over 4.8 GB of VRAM free.

Mathematical Formulation of Focal Tversky Loss

Because active wildfires represent only 0.22% of the spatial pixels in Northern Algeria, optimizing with standard Binary Cross-Entropy (BCE) causes the network to converge to a degenerate local minimum where it predicts 0.0 everywhere.

To overcome this, we train using **Focal Tversky Loss**.

```
Confusion Matrix Continuous Relaxation:
  Ground Truth G  {0, 1}
  Model Sigmoid Output P  [0.0, 1.0]

            G = 1 (Fire)           G = 0 (Background)

P -> 1    True Positive (TP)   False Positive (FP)
(Fire)        P * G                  P * (1 - G)

P -> 0   False Negative (FN)    True Negative (TN)
(Safe)     (1 - P) * G           (1 - P) * (1 - G)
```

1. Continuous Tversky Index

Let $p_{bi} \in [0.0, 1.0]$ denote the predicted probability of fire for pixel i in batch item b (obtained via $\sigma(\text{logit})$), and let $g_{bi} \in \{0.0, 1.0\}$ represent the ground truth label. We define the continuous relaxations of True Positives (TP), False Positives (FP), and False Negatives (FN):

$$TP = \sum_{i=1}^{H \times W} p_i g_i, \quad FP = \sum_{i=1}^{H \times W} p_i(1 - g_i), \quad FN = \sum_{i=1}^{H \times W} (1 - p_i) g_i$$

The Tversky Index (TI) generalizes the Dice coefficient (F_1-score) by decoupling the penalties assigned to false alarms versus missed detections:

$$TI = \frac{TP + \epsilon}{TP + \alpha FP + \beta FN + \epsilon}$$

- $\epsilon = 10^{-6}$ is a smoothing factor preventing division by zero when an entire frame has no active fire.
- α is the penalty weight for False Positives (false alarms).
- β is the penalty weight for False Negatives (missed fires).

We enforce $\alpha + \beta = 1.0$. Setting $\alpha = 0.3$ and $\beta = 0.7$ weights missed wildfires more heavily than false alarms, pushing gradient updates toward detecting faint thermal signatures.

2. The Focal Modulation

Standard Tversky loss is linear: $\mathcal{L}_{TL} = 1 - TI$. To prevent easy, obvious non-fire backgrounds from dominating training, we apply a focal exponent γ:

$$\mathcal{L}_{FTL} = (1 - TI)^{1/\gamma}$$

When $\gamma = 4/3 \approx 1.33$, the exponent $1/\gamma = 0.75$ steepens the loss gradient when $TI < 0.5$, penalizing poorly segmented fire boundaries while preventing well-segmented fires ($TI \to 1.0$) from overwhelming the backpropagation updates.

PyTorch Implementation: Losses and Segmentation Metrics

Create a new file named `losses_and_metrics.py` in your project workspace:

```
import torch
import torch.nn as nn


class FocalTverskyLoss(nn.Module):
  """Focal Tversky Loss for highly imbalanced binary semantic
     segmentation.

  Formula:
      TI = (TP + eps) / (TP + alpha * FP + beta * FN + eps)
      FTL = (1 - TI) ** (1 / gamma)
  """

  def __init__(self, alpha=0.3, beta=0.7, gamma=4.0 / 3.0, eps=1e
     -6):
    super(FocalTverskyLoss, self).__init__()
    self.alpha = alpha
    self.beta = beta
    self.gamma = gamma
    self.eps = eps

  def forward(self, logits, targets):
    """Args:

        logits: Tensor of shape [B, 1, H, W] (Unconstrained raw
           model outputs)
        targets: Tensor of shape [B, 1, H, W] (Binary masks with
           values 0.0 or
        1.0)
    """
    # 1. Map raw logits to continuous probabilities via Sigmoid
       activation
    probs = torch.sigmoid(logits)

    # 2. Flatten spatial dimensions per batch item: [B, H*W]
    probs_flat = probs.view(probs.size(0), -1)
    targets_flat = targets.view(targets.size(0), -1)

    # 3. Compute continuous confusion matrix terms
    true_pos = torch.sum(probs_flat * targets_flat, dim=1)
    false_pos = torch.sum(probs_flat * (1.0 - targets_flat), dim
       =1)
    false_neg = torch.sum((1.0 - probs_flat) * targets_flat, dim
       =1)

    # 4. Compute Tversky Index per batch item
```

```
    tversky_index = (true_pos + self.eps) / (
        true_pos
        + self.alpha * false_pos
        + self.beta * false_neg
        + self.eps
    )

    # 5. Apply Focal modulation and average across batch
    focal_tversky = torch.pow((1.0 - tversky_index), (1.0 / self.
       gamma))
    return torch.mean(focal_tversky)


#
   ============================================================================

# EVALUATION METRICS (Binarized via Threshold = 0.5)
#
   ============================================================================

@torch.no_grad()
def calculate_segmentation_metrics(logits, targets, threshold
   =0.5, eps=1e-6):
  """Computes Dice (F1), IoU (Jaccard), Precision, and Recall on
     binarized predictions.

  Args:
      logits: Tensor of shape [B, 1, H, W]
      targets: Tensor of shape [B, 1, H, W]
  Returns:
      dict: Averaged scalar metrics across the batch
  """
  probs = torch.sigmoid(logits)
  preds = (probs > threshold).float()

  preds_flat = preds.view(preds.size(0), -1)
  targets_flat = targets.view(targets.size(0), -1)

  tp = torch.sum(preds_flat * targets_flat, dim=1)
  fp = torch.sum(preds_flat * (1.0 - targets_flat), dim=1)
  fn = torch.sum((1.0 - preds_flat) * targets_flat, dim=1)

  # Dice Score / F1: 2*TP / (2*TP + FP + FN)
  dice = (2.0 * tp + eps) / (2.0 * tp + fp + fn + eps)

  # IoU / Jaccard Index: TP / (TP + FP + FN)
  iou = (tp + eps) / (tp + fp + fn + eps)

  # Precision: TP / (TP + FP)
  precision = (tp + eps) / (tp + fp + eps)
```

```
  # Recall / Sensitivity: TP / (TP + FN) -> Crucial for wildfire
      detection
  recall = (tp + eps) / (tp + fn + eps)

  return {
      "dice": torch.mean(dice).item(),
      "iou": torch.mean(iou).item(),
      "precision": torch.mean(precision).item(),
      "recall": torch.mean(recall).item(),
  }
```

Verification and Sanity Testing

To confirm the loss calculation and evaluation metrics behave as expected, create and execute `test_loss.py`:

```
import torch
from losses_and_metrics import FocalTverskyLoss,
    calculate_segmentation_metrics

criterion = FocalTverskyLoss(alpha=0.3, beta=0.7)

# 1. Create a dummy ground truth mask with sparse fires (
    simulating 66 fire pixels)
targets = torch.zeros(4, 1, 128, 256)
targets[:, :, 20:25, 200:215] = 1.0  # Fire patch

# Case A: Perfect Predictions (Large positive logits -> Sigmoid
    -> ~1.0 on fires)
perfect_logits = torch.full_like(targets, -10.0)  # Background =
    0.0
perfect_logits[:, :, 20:25, 200:215] = 10.0  # Fire = 1.0
loss_perfect = criterion(perfect_logits, targets)
metrics_perfect = calculate_segmentation_metrics(perfect_logits,
    targets)

# Case B: Degenerate "All Background" Model (Outputs zero fires
    everywhere)
all_zeros_logits = torch.full_like(targets, -10.0)
loss_zeros = criterion(all_zeros_logits, targets)
metrics_zeros = calculate_segmentation_metrics(all_zeros_logits,
    targets)

# Case C: Completely Inverted Predictions (Predicts fire where
    there is none)
inverted_logits = torch.full_like(targets, 10.0)
inverted_logits[:, :, 20:25, 200:215] = -10.0
loss_inverted = criterion(inverted_logits, targets)
metrics_inverted = calculate_segmentation_metrics(inverted_logits
    , targets)
```

```
print("--- Sanity Check: Focal Tversky Loss & Metrics ---")
print(
    f"Case A (Perfect Match)  -> Loss: {loss_perfect.item():.5f}
       | Dice:"
    f" {metrics_perfect['dice']:.4f} | Recall: {metrics_perfect['
       recall']:.4f}"
)
print(
    f"Case B (All Background) -> Loss: {loss_zeros.item():.5f} |
       Dice:"
    f" {metrics_zeros['dice']:.4f} | Recall: {metrics_zeros['
       recall']:.4f}"
)
print(
    f"Case C (Fully Inverted) -> Loss: {loss_inverted.item():.5f}
        | Dice:"
    f" {metrics_inverted['dice']:.4f} | Recall:"
    f" {metrics_inverted['recall']:.4f}"
)

# Mathematical assertions
assert (
    loss_perfect < 0.05
), "Loss on perfect predictions must be close to zero!"
assert (
    loss_zeros > 0.80
), "Loss on degenerate all-background predictions must be heavily
    penalized!"
assert (
    metrics_perfect["dice"] > 0.99
), "Dice score on perfect predictions must be ~1.0!"
assert (
    metrics_zeros["recall"] < 0.01
), "Recall on all-zero predictions must be 0.0!"
print("\nAll mathematical constraints validated successfully.")
```

Run this script in your terminal:

```
python test_loss.py
```

Would you like to assemble the full `train.py` script now with the AdamW optimizer, validation tracking, and model checkpoint saving?